MONITOR Z · L15.5

Clase 5 — Cómo se llaman los módulos: la convención de enlace

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L15 · Clase 5 — Cómo se llaman los módulos: la convención de enlace

En la Clase 4 viste que una llamada externa deja un hueco V, y que el binder lo llena. Esta clase explica qué pasa **en el momento de la llamada**: cómo pasa el control de un módulo a otro, cómo le pasa los datos y cómo vuelve sin romper nada. Es un contrato fijo de z/OS, y ZTKMON ya lo cumple dos veces: cuando z/OS lo llama a él y cuando él llama a `EZASOKET`.

> 📚 Fuentes: el listado del ensamblado de **JOB34775**: el prólogo (sentencias 10126–10137), el epílogo `CERRAR` (10320–10334), el `CALL EZASOKET` de `ABRESOCK` (11305–11326) y la `SAVEAREA` (12700). *z/OS MVS Programming: Assembler Services Guide* (SA23-1368), cap. «Linkage conventions». Ver también la *L01*.

1. El contrato: cuatro registros con un trabajo fijo

Registro	Al llamar, lo carga…	Con qué	Al volver
R15	el que llama	la **dirección de entrada** del llamado (el hueco V)	el llamado pone el **código de retorno**: 0 bien, 4 aviso, 8 error
R14	el que llama	la **dirección de vuelta**	el llamado salta ahí: `BR R14`
R1	el que llama	la dirección de la **lista de parámetros**	—
R13	el que llama	la dirección de **su** *save area*	igual que antes

El resto (R0, R2–R12) el llamado los tiene que devolver como los encontró. Por eso los guarda apenas entra.

Analogía: una carrera de relevos. El que entrega la posta dice adónde volver (R14), quién la recibe (R15), qué le pasa (R1) y dónde anotar lo que tenía en las manos (R13). El que la recibe anota todo antes de correr y, al terminar, deja todo como estaba y avisa cómo le fue (R15).

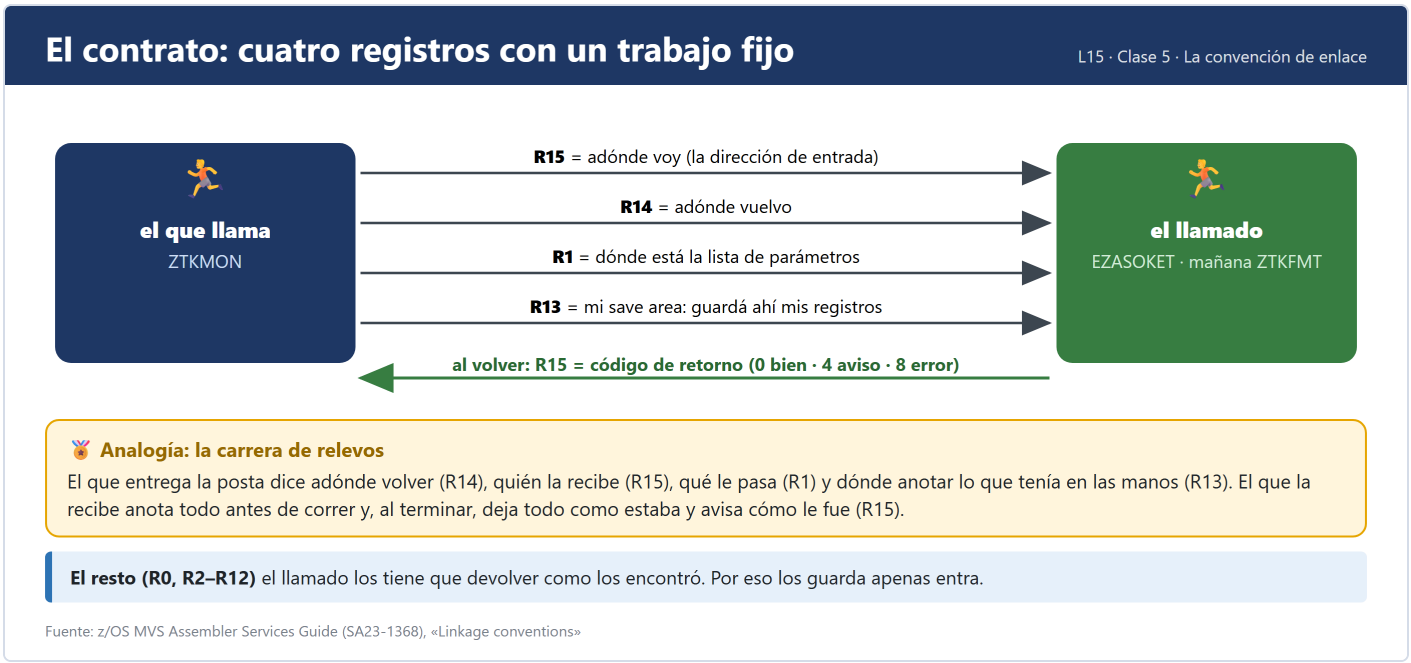


Lámina L15-18_el_contrato

2. La *save area*: 18 palabras para guardar los registros

ZTKMON tiene la suya: `SAVEAREA DS 18F`, en X'1F88' (sentencia 12700). Son 72 bytes con esta forma:

Desplazamiento	Qué guarda	Quién lo escribe
+0	(lo usa el lenguaje; ZTKMON no lo toca)	—
+4	la dirección de la *save area* **del que me llamó** (hacia atrás)	el llamado
+8	la dirección de la *save area* **del que yo llamo** (hacia adelante)	el llamado
+12	R14	el llamado (`STM`)
+16	R15	el llamado (`STM`)
+20 … +68	R0 a R12	el llamado (`STM`)

Regla clave: el llamado guarda los registros del que llamó **en la *save area* del que llamó**, la que le llega en R13. Su *save area* propia es para cuando él, a su vez, llame a otro.

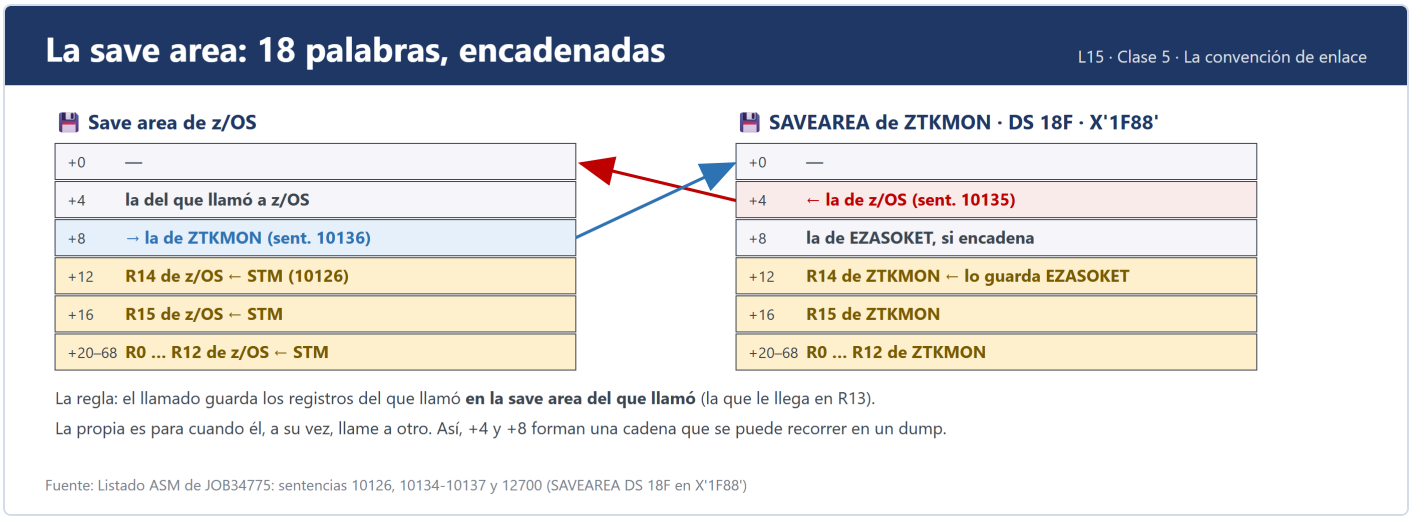


Lámina L15-19_save_area

3. ZTKMON como llamado: su prólogo y su epílogo

Cuando z/OS arranca ZTKMON, ZTKMON es el **llamado**. Así cumple el contrato (JOB34775):

Al entrar:

Sent.	Fuente	Bytes	Qué hace
10126	`STM R14,R12,12(R13)`	`90EC D00C`	guarda R14, R15 y R0–R12 en la *save area* de z/OS (R13), desde +12
10127	`LR R12,R15`	`18CF`	**R15 trae la dirección de entrada**: es la base (Clase 3)
10134	`LA R2,SAVEAREA`	`4120 BF88`	R2 = mi *save area* (X'1F88')
10135	`ST R13,4(,R2)`	`50D0 2004`	en la mía, +4: la de z/OS (hacia atrás)
10136	`ST R2,8(,R13)`	`5020 D008`	en la de z/OS, +8: la mía (hacia adelante)
10137	`LR R13,R2`	`18D2`	desde ahora R13 = mi *save area*, lista para cuando yo llame

Al salir (`CERRAR`):

Sent.	Fuente	Bytes	Qué hace

10329	`L R15,RCFINAL`	`58F0 BE38`	R15 = el código de retorno
10331	`L R13,4(,R13)`	`58D0 D004`	vuelve a la *save area* de z/OS (el +4 de la mía)
10332	`L R14,12(,R13)`	`58E0 D00C`	recupera la dirección de vuelta
10333	`LM R0,R12,20(R13)`	`980C D014`	recupera R0–R12. **No toca R15**: ahí va el RC
10334	`BR R14`	`07FE`	vuelve a z/OS

Además, ZTKMON guarda su `PARM` antes de usar R1: `ST R1,SALVAR1` (sentencia 10133). Es el R1 de la tabla: la lista de parámetros que le pasó z/OS.

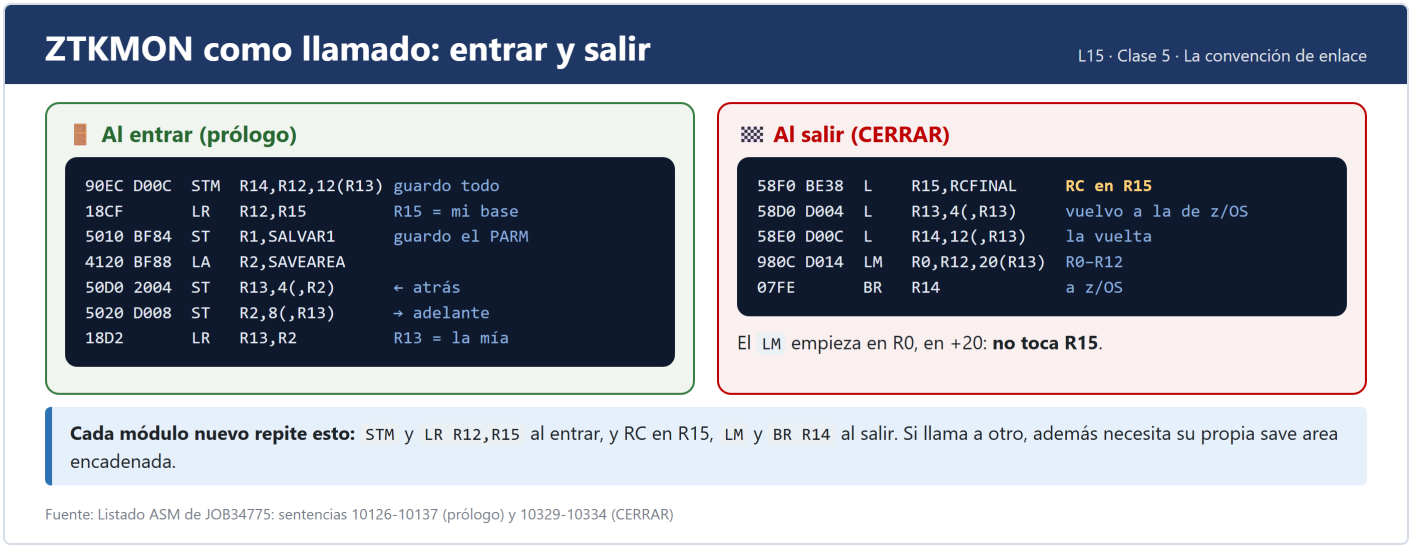


Lámina L15-20_prologo_epilogo

4. ZTKMON como llamador: el `CALL EZASOKET`

Cuando ZTKMON llama a `EZASOKET`, los papeles se invierten. El `CALL` de `ABRESOCK` (sentencias 11307–11326) se expande así:

Loc	Bytes	Fuente	Qué hace
0DD8	`00000000`	`DC V(EZASOKET)`	el hueco que llena el binder (Clase 4)
0DDC	`4110 CDE4`	`LA 1,IHB0032`	**R1** = dirección de la lista

0DE4–0DFC	7 palabras	`DC A(FINITAPI)` … `DC A(RETCODE+X'80000000')`	la **lista de parámetros**: una dirección por parámetro
0E00	`58F0 CDD8`	`L 15,IHB0030B`	**R15** = la dirección de EZASOKET (lo que hay en el hueco)
0E04	`05EF`	`BALR 14,15`	**R14** = la vuelta (X'0E06') y salta a R15

R13 ya apunta a la `SAVEAREA` de ZTKMON desde el prólogo: `EZASOKET` guarda ahí los registros de ZTKMON.

La lista de parámetros lleva una dirección por parámetro, no los datos. La última viene marcada:

Loc	Bytes	Parámetro
0DE4	`00001FD0`	`FINITAPI`
0DE8	`00002070`	`MAXSOC`
…	…	…
0DF8	`000020A0`	`ERRNO`
0DFC	**`800020A4`**	`RETCODE` , el **último**: tiene prendido el **bit alto**

Ese bit es lo que pide el `,VL` del `CALL` (*variable list*): así el llamado sabe dónde termina la lista sin que nadie le diga cuántos parámetros son. Funciona porque con `AMODE 31` una dirección usa sólo 31 de los 32 bits (Clase 2).

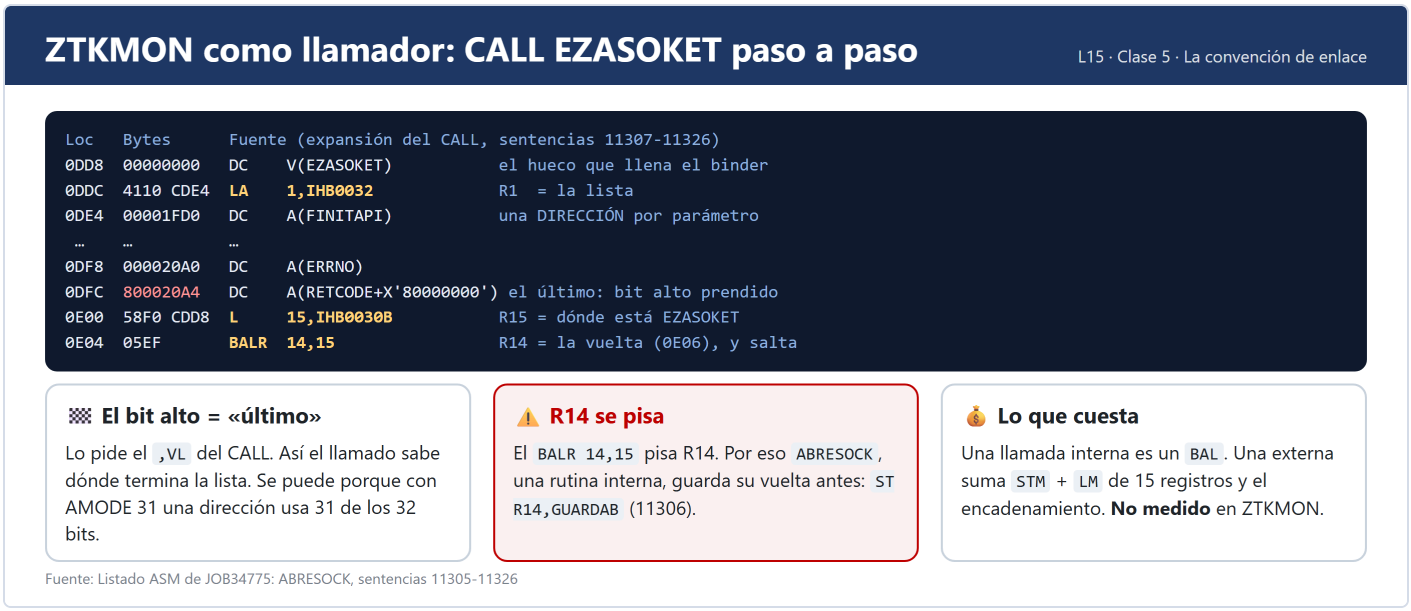


Lámina L15-21_call_paso_a_paso

5. Las rutinas internas no usan el contrato completo

`BAL R14,AHORA` (Clase 4) sólo usa R14 para volver: no guarda registros ni encadena *save areas*. Por eso cuesta poco. El problema aparece cuando una rutina interna llama a otra cosa: en `ABRESOCK` el `BALR 14,15` del `CALL` **pisa R14**, y la vuelta se perdería. Por eso la rutina lo guarda antes: `ST R14,GUARDAB` (sentencia 11306).

	Rutina interna (`BAL`)	Módulo externo (`CALL`)
Para volver	R14	R14
Guarda registros	no (cada una cuida R14 a mano, `GUARDAB` …)	sí: `STM` en la *save area* del que llama
Encadena *save areas*	no	sí: +4 y +8
RC en R15	por costumbre (ej. `ABRESOCK` : 0 u 8)	**por contrato**
Costo	1 instrucción	`STM` + `LM` de 15 registros y el encadenamiento. **No medido** en ZTKMON

6. Qué cambia con los módulos

Cada módulo nuevo (`ZTKFMT`, `ZTKJES`…) es un **llamado**, y repite el prólogo y el epílogo de la sección 3:

- `STM R14,R12,12(R13)` al entrar, y `LR R12,R15` para su base. Es el mismo truco de ZTKMON: R15 trae dónde empieza.
- Si a su vez llama a otro, necesita **su propia *save area*** y encadenarla.
- Al salir: RC en R15, `LM` y `BR R14`.
- Los datos que necesita le llegan por **R1**, con una lista de parámetros, o por el área común (Clase 6).

La pregunta abierta para la Clase 6: ¿dónde vive la *save area* de cada módulo? Si está adentro de la CSECT, como hoy `SAVEAREA`, el módulo **no es reentrante**: escribe en sí mismo.

7. Resumen

Idea	En una línea
R15	al llamar, adónde voy; al volver, cómo me fue (RC)
R14	adónde vuelvo
R1	dónde está la lista de parámetros: una dirección por parámetro; la última, con el bit alto
R13	la *save area* del que llama: ahí el llamado guarda los registros
Save area	18 palabras: +4 hacia atrás, +8 hacia adelante, +12 R14, +16 R15, +20 R0–R12
Prólogo y epílogo	`STM` y encadenar al entrar; RC en R15, `LM` y `BR R14` al salir

8. Preguntas de comprobación

1. En la lista de parámetros del `CALL EZASOKET`, la última dirección es `800020A4` y no `000020A4`. ¿Por qué, y qué parte del `CALL` lo pide?
2. Al salir, ZTKMON hace `L R15,RCFINAL` y después `LM R0,R12,20(R13)`, que no toca R15. ¿Qué recibe z/OS en R15, y qué pasaría si el `LM` fuera `LM R14,R12,12(R13)`?

Mis respuestas

Notas